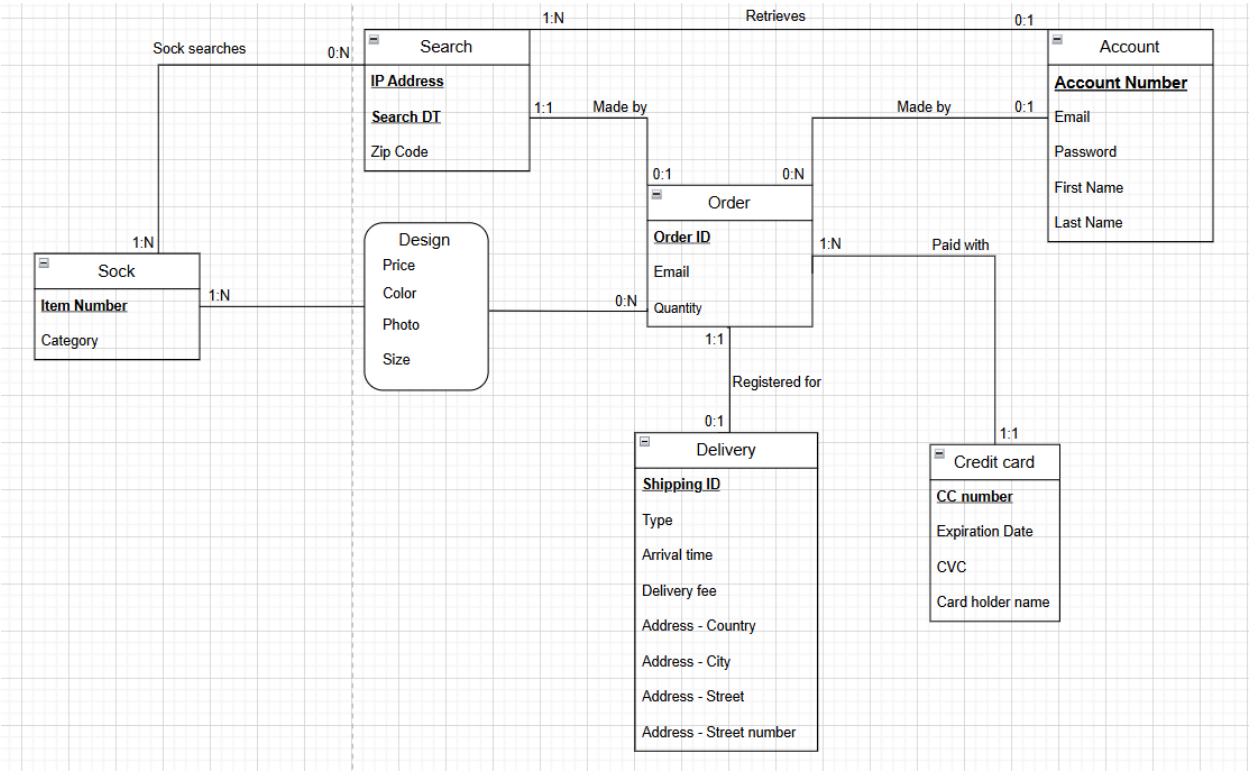
מס' קבוצה	אתר	תאריך הגשה
28	https://sockologie.com/	15/06/2025
חברי הצוות - מספרי ת.ז		
208279679	207531120	206974800

פרויקט בסיסי נתונים – חלק ג'

פרק ראשון – תיקון ה-ERD :

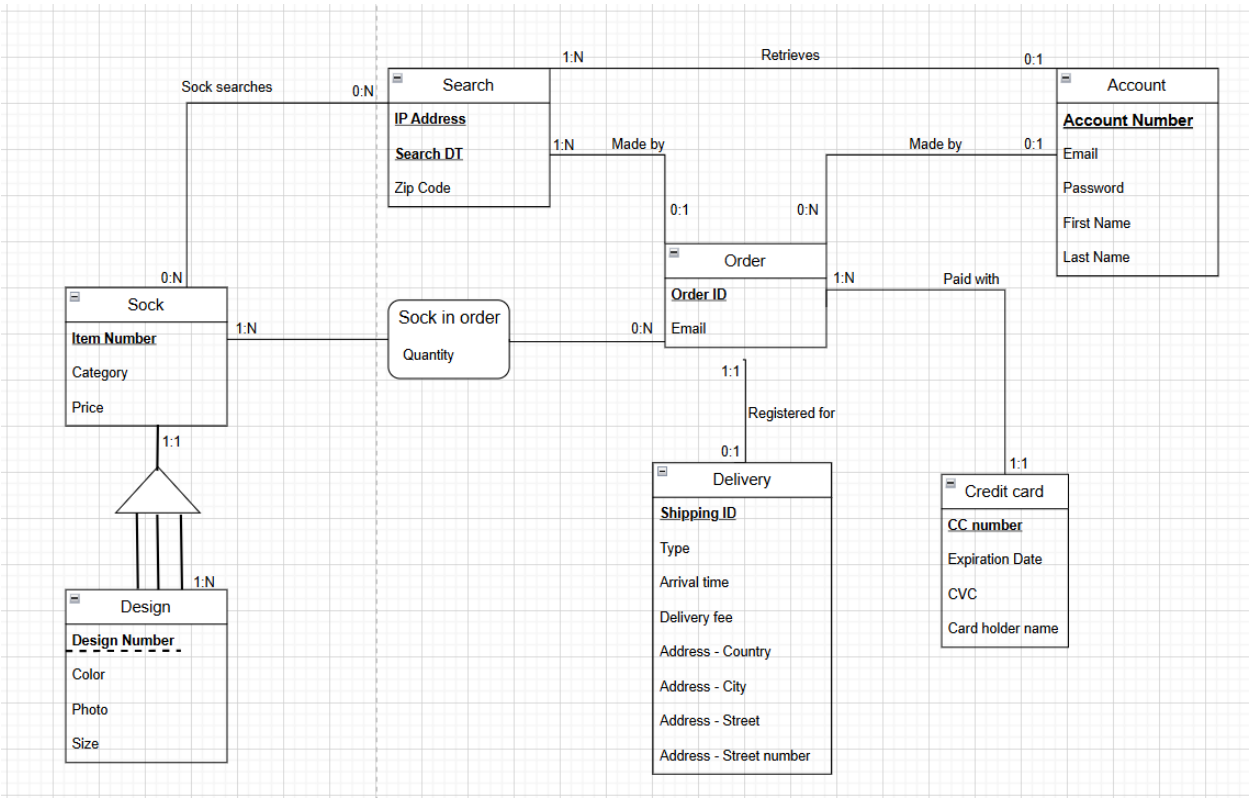
להלן מודל ה-ERD מחלק ב לפניי התיקונים :



רשימת ההערות שקיבלנו בהגשה הקודמת :

1. "הזמנה יכולה להגיע מכמה חיפושים שונים"
2. "לא כל גרב חייבת להיות מחופשת"
3. "כמות לא יכולה להיות על הזמנה מכיוון שאפשר להזמין גרביים שונות באותה ההזמנה. איך נדע על איזה גרב זה מדבר?"
4. "עיצוב לא יכול להיות בקשר בין הזמנה למוצר. תחשבו מה קורה אם באותה הזמנה אנו מזמינים אותו גרב בעיצובים שונים? כל דבר שעל הקשר ימחק".

להלן מודל ה-ERD לאחר התיקונים על ההערות שקיבלנו בחלק ב' :



ובהתאמה תיקון העיצוב הלוגי ש-"מטפל" בכל ההערות שקיבלנו :

- **Socks** (Item Number , Category, Price)
- **Sock Searches** (Item Number (Socks) , {IP Address , Search DT} (Searches))
- **Searches** (IP Address , Search DT **,** Zip Code , Account Number (Accounts) , Order ID (Orders))
- **Orders** (Order ID , Email , Account Number (Accounts) , CC Number (Credit Cards))
- **Accounts** (Account Number , Email , Password , First Name , Last Name)
- **Credit Cards** (CC Number , Expiration Date , CVC , Card holder name)
- **Designs** (Item Number (Sock) , Design Number , Color ,
- Photo , Size)
- **Deliveries** (Shipping ID **,** Type , Arrival time , Delivery fee , Country , City , Street , Street Number, Order ID (Orders))
- **Sock in Order** (Item Number (Socks) , Order ID (Orders) , Quantity)

להלן רשימת השינויים שבוצעו בקוד טרם הכנסת הנתונים על מנת להתאים את הבסיס נתונים למודל ה-ERD המתוקן:

הוספת שדה Price לטבלת Sock:

ALTER TABLE Socks ADD Price MONEY NOT NULL

הסרת שדה Quantity מטבלת Orders כולל אילוצים רלוונטים:

ALTER TABLE Orders DROP CONSTRAINT CK_Quantity

ALTER TABLE Orders DROP COLUM Quantity

הסרת השדות Price ו-Order_ID מטבלת Designs:

ALTER TABLE Designs DROP COLUM Price

ALTER TABLE Designs DROP CONSTRAINT FK_Designs_Order_ID

ALTER TABLE Designs DROP CONSTRAINT PK_Designs

ALTER TABLE Designs DROP COLUM Order_ID

הוספת שדה DesignNumber לטבלת Design:

ALTER TABLE Designs ADD DesignNumber INT NOT NULL

הסרת מפתח ראשי קודם והוספת מפתח ראשי חדש לטבלת Designs:

ALTER TABLE Designs ADD CONSTARINT PK_Designs PRIMARY KEY (Item_Number , DesignNumber)

יצירת טבלה חדשה בשם SockInOrder:

```
CREATE TABLE SockInOrder(
        Item_Number        INT     NOT NULL
        Order_ID           INT     NOT NULL
        Quantity           INT     NOT NULL
        CONSTRAINT PK_SockInOrder PRIMARY KEY (Item_Number , Order_ID),
        CONSTRAIN FK_SockInOrder_Item FOREIGN KEY (Item_Number) REFERENCES Sock(Item_Number),
        CONSTRAINT FK_SockInOrder_Order FOREIGN KEY (Order_ID) REFERENCES Orders (Order_ID)
        )
```

פרק ראשון – מטלות חובה

מטלה 1 – שאילתות :

שתי שאילתות SELECT ללא קינון –

שאילתה ללא קינון מספר 1:

קוד ה- SQL –

```
SELECT sc.Category_Name AS Category, SUM (so.Quantity) AS Total_Sold

FROM SockInOrder AS so JOIN Socks AS s ON so.Item_Number = s.Item_Number JOIN Sock_Categories AS sc ON s.Category = sc.Category_Name

WHERE S.Category in ('Junk' , 'Food' , 'Christmas' , 'Funny' , 'Halloween' , 'Wine' , 'Winter')

GROUP BY sc.Category_Name

ORDER BY Total_Sold DESC
```

הגיון עסקי – בחודש פברואר אשתקד מחלקת השיווק הוסיפה 6 קטגוריות חדשות בבחירה של עיצוב גרביים לאתר, מטרת השאילתה הינה לבדוק מה "סטטוס" המכירות שלהם.

צילום מסך מהפלט שמפיק הבסיס נתונים:

	Category	Total_Sold
1	Junk Food	131
2	Christmas	28
3	Halloween	26
4	Winter	19
5	Funny	10
6	Wine	9

שאילתה ללא קינון מספר 2:

קוד ה – SQL –

```
SELCET D.Country , COUNT(*) AS Pickup_Count

FROM Deliveries  AS D

WHERE D.[Type] = 'Pickup in store'

GROUP BY D.Country

HAVING COUNT(*) <= 4

ORDER BY Pickup_Count
```

הגיון עסקי – אנו מבצעים בדיקה באילו מן המדינות שבהם ניתן לאסוף בצורה עצמאית שיעור האיסופיים העצמאיים נמוך כך נוכל לפעול בצמצום מקומות או ביטולם לגמרי ובכך להתייעל כלכלית.

צילום מסך מהפלט שמפיק הבסיס נתונים :

	Country	Pickup_Count
1	Italy	2
2	Japan	2
3	Netherlands	3
4	Australia	3
5	Canada	3
6	Israel	3

שתי שאילתות SELECT מקוננות :

שאילתה מקוננת מספר 1 (שאילתה טבלאית) :

קוד ה – SQL –

```
SELECT d.County , YEAR(d.Arrival_Timer) AS Delivery_Year , AVG(order_total.Total_Amount) AS Avg_Order_Amount, COUNT(DISTINCT d.Order_ID) AS Total_Orders

FROM Deliveries AS d JOIN Orders AS o ON d.Country = c.Cpuntry_Name, JOIN( SELECT so.Order_ID , SUM(so.Quantity * s.Price) AS Total_Amout FROM SockInOrder AS so JOIN Sock AS s ON so.Item_Number = s.Item_Number GROUP BY so.Order_ID) AS order_total ON d.Order_ID = order_total.Order_ID

GROUP BY d.Country, YEAR(d.Arrival_Timr)

ORDER BY d.County,Delivry_Year
```

הגיון עסקי – מה ממוצע התשלום של לקוחות פר שנה לפי מדינה

צילום מסך מהפלט שמפיק הבסיס נתונים:

	Country	Delivery_Year	Avg_Order_Amount	Total_Orders
1	Australia	2025	67.1052	19
2	Canada	2025	54.7894	19
3	France	2025	59.8421	19
4	Germany	2025	63.0526	19
5	Israel	2025	71.5789	19
6	Italy	2025	54.3809	21
7	Japan	2025	51.30	20
8	Netherl...	2025	53.2352	17
9	Spain	2025	56.10	20
10	USA	2025	58.8846	26

שאילתה מקוננת מספר 2 (שאילתה סקלרית) :

קוד ה – SQL –

```
SELECT (SELECT COUNT(*) , FROM Sock_Searches , WHERE MONTH(Search_DT) = 3 AND YEAR(Search_DT) = 2025) AS March_Searches , SELECT COUNT (*) / COUNT(DISTINCT MONTH(Search_DT)) , FROM Sock_Searches , WHERE YEAR(Search_DT) = 2025) AS Avg_Searches_Per_Month

CASE WHEN ((SELECT COUNT(*) , FROM Sock_Searches , WHERE MONTH(Search_DT) = 3 AND YEAR(Search_DT) = 2025)

(SELECT COUNT(*) / COOUNT(DISTINCT MONTH(Search_DT)) , FROM Sock_Searches , WHERE YEAR (Search_DT) = 2025))>=0 THEN '+' + CAST(FLOOR(100.0*(SELECT COUNT(*) , FROM Sock_Searches , WHERE MONTH(Search_DT) = 3 AND YEAR(Search_DT) = 2025) (SELECT COUNT(*) / COUNT(DISTINCT MONTH(Search_DT)) , FROM Sock_Searches , WHERE YEAR(Search_DT) = 2025)) / NULLIF ((SELECT COUNT(*) / COUNT(DISTINCT MONTH (Search_DT)) , FROM Sock_Searches , WHERE YEAR (SEARCH_DT) = 2025) , 0)) AS VARCHAR) +'%'

ELSE '-' + CAST(FLOOR(100.0*ABS(SELECT COUNT(*) , FROM Sock_Searches , WHERE MONTH(Search_DT) = 3 AND YEAR(Search_DT) = 2025)

(SELECT COUNT(*) / COUNT(DISTICNT MONTH(Search_DT) = 2025)

(SELECT COUNT(*) / COUNT(DISTICNT MONTH(Search_DT) , FROM Sock_Searches ,

WHERE YEAR(Search_DT) = 2025)) / NULLIF ((SELECT COUNT(*) / COUNT(DISTINCT MONTH(Search_DT)) ,

FROM Sock_Searches ,

WHERE YEAR(Search_DT) = 2025),0) AS VARCHAR) + '%' END AS Change_March_vs_Avg
```

הגיון עסקי – מחלקת השיווק יצאה בקמפיין גדול בחודש פברואר, לאור זה רצינו לבדוק את המכירות של חודש מרץ כלומר את ההשפעה של הקמפיין על המכירות ביחס לחודשים אחרים.

צילום מסך מהפלט שמפיק הבסיס נתונים :

	March_Searches	Avg_Searches_Per_Month	Change_March_vs_Avg
1	140	100	+40%

שאילתות עסקיות המשלבות Window Functions :

שאילתת חלון מספר 1 המכילה 2 פונקציות :

קוד ה – SQL –

```
SELECT D.Country , S.Item_Number,

    SUM(SO.Quantity) AS Total_Units_Sold,

    RANK() OVER (PARTITION BY D.Country ORDER BY SUM(SO.Quantity) DESC) AS Sock_Rank,

    FIRST_VALUE(SUM(SO.Quantity)) OVER (PARTITION BY D.Country ORDER BY SUM(SO.Quantity) DESC)
AS Top_Sales,

    FIRST_VALUE(SUM(SO.Quantity)) OVER (PARTITION BY D.Country ORDER BY SUM(SO.Quantity) DESC)

 -  SUM(SO.Quantity) AS Sales_Difference

FROM  SockInOrder  AS SO JOIN Deliveries AS D ON SO.Order_ID = D.Order_ID JOIN Socks AS S ON
SO.Item_Number = S.Item_Number

GROUP BY D.Country, S.Item_Number

ORDER BY D.Country, Sock_Rank
```

הגיון עסקי – הפונקציה מציגה לכל מדינה את סך הגרביים שנמכרו עבור כל סוג גרב ומדרגת אותם על פי הגרב הנמכרת ביותר עד לגרב הכי פחות פופולרית בנוסף הפונקציה מציגה את ההפרש בכמות הגרביים הנמכרות בין הדגם הפופולרי ביותר להכי פחות פופולרי על מנת לתת למחלקת השיווק מושג עסקי לגבי מה כדאי לשווק בצורה אגרסיבית יותר.

צילום מסך מהפלט שמפיק בסיס הנתונים –

	Country	Item_Number	Total_Units_Sold	Sock_Rank	Top_Sales	Sales_Difference
19	Australia	10011	3	8	7	4
20	Australia	10009	2	20	7	5
21	Australia	10048	2	20	7	5
22	Australia	10042	1	22	7	6
23	Australia	10033	1	22	7	6
24	Australia	10037	1	22	7	6
25	Australia	10000	1	22	7	6
26	Australia	10024	1	22	7	6
27	Australia	10030	1	22	7	6
28	Canada	10010	6	1	6	0
29	Canada	10027	5	2	6	1
30	Canada	10018	5	2	6	1
31	Canada	10038	5	2	6	1
32	Canada	10045	4	5	6	2
33	Canada	10049	4	5	6	2

שאילתת חלון מספר 4 המכילה 2 פונקציות –

קוד ה- SQL -

```
SELECT S.Item_Number,
 CONCAT(YEAR(SS.Search_DT), '-', RIGHT('0' + CAST(MONTH(SS.Search_DT) AS VARCHAR(2)), 2)) AS Search_Month,
 COUNT(*) AS Monthly_Searches,
 LAG(COUNT(*)) OVER (PARTITION BY S.Item_Number ORDER BY YEAR(SS.Search_DT), MONTH(SS.Search_DT)) AS Prev_Month_Searches,
 COUNT(*) - LAG(COUNT(*)) OVER (PARTITION BY S.Item_Number ORDER BY YEAR(SS.Search_DT), MONTH(SS.Search_DT)) AS Monthly_Change,
SUM(COUNT(*)) OVER (PARTITION BY S.Item_Number ORDER BY YEAR(SS.Search_DT), MONTH(SS.Search_DT)) AS Cumulative_Searches
FROM Sock_Searches AS SS JOIN Socks AS S ON SS.Item_Number = S.Item_Number
GROUP BY S.Item_Number, YEAR(SS.Search_DT), MONTH(SS.Search_DT)
ORDER BY S.Item_Number, Search_Month;
```

הגיון עסקי – בדיקה עבור כל גרב מה הסטטוס חיפושים שלה החודש אל מול החודש הקודם במטרה לזהות שינויי מגמה ולקבל אינדיקציה כדאי לפעול בשינויים דרסטיים.

צילום מסך מהפלט שמפיק בסיס הנתונים –

	Item_Number	Search_Month	Monthly_Searches	Prev_Month_Searches	Monthly_Change	Cumulative_Searches
1	10000	2025-03	2	NULL	NULL	2
2	10000	2025-04	3	2	1	5
3	10000	2025-05	4	3	1	9
4	10000	2025-06	5	4	1	14
5	10001	2025-03	5	NULL	NULL	5
6	10001	2025-04	1	5	-4	6
7	10001	2025-06	1	1	0	7
8	10002	2025-03	5	NULL	NULL	5
9	10002	2025-04	4	5	-1	9
10	10002	2025-05	2	4	-2	11
11	10002	2025-06	5	2	3	16
12	10003	2025-03	2	NULL	NULL	2
13	10003	2025-04	3	2	1	5
14	10003	2025-05	1	3	-2	6
15	10004	2025-03	4	NULL	NULL	4
16	10004	2025-04	2	4	-2	6
17	10004	2025-05	3	2	1	9

שאילתה מקוננת תוך שימוש ב-CTE :

קוד ה – SQL –

```
WITH OrderTotalsPerCity AS (

SELECT D.City,

COUNT(D.Order_ID) AS Num_Orders,

SUM(S.Price * SO.Quantity) AS Total_Value,

AVG(S.Price * SO.Quantity) AS Avg_Order_Value

FROM Deliveries AS D JOIN Orders AS O ON D.Order_ID = O.Order_ID JOIN SockInOrder AS SO ON
O.Order_ID = SO.Order_ID JOIN Socks AS S ON SO.Item_Number = S.Item_Number

GROUP BY D.City ) ,

CityStats AS(

SELECT    AVG(Num_Orders * 1.0) AS Avg_Orders,

AVG(Avg_Order_Value * 1.0) AS Avg_Order_Value

FROM OrderTotalsPerCity

) ,
```

```
HighValueCities AS (
SELECT OC.Cit ,  OC.Num_Orders , OC.Total_Value , OC.Avg_Order_Value
FROM OrderTotalsPerCity AS OC CROSS JOIN CityStats AS CS
WHERE OC.Num_Orders >= CS.Avg_Orders AND OC.Avg_Order_Value >= CS.Avg_Order_Value
)
SELECT    City , Num_Orders , Total_Value , Avg_Order_Value
FROM HighValueCities
ORDER BY Total_Value DESC
```

הגיון עסקי – אילו ערים יצרו את ההזמנות הרווחיות ביותר כלומר ערים בהן סך שווי ההזמנה גבוה בהשוואה לממוצע אנחנו רוצים למצוא ערים שיש בהן כמות משמעותית של הזמנות ושההזמנות בהן מכניסות סכום גבוה במיוחד כך שערים אלה מייצרות גם פעילות גדולה וגם ערך גבוה. המטרה היא לספק למחלקת שיווק מידע שיעזור להם לקבל החלטות על נקודות איסוף עצמי שיוזיל עלויות שילוח.

צילום מסך מהפלט שמפיק הבסיס נתונים –

	City	Num_Orders	Total_Value	Avg_Order_Value
1	New York	15	640.00	42.6666
2	Haifa	14	613.00	43.7857
3	Madrid	14	566.00	40.4285
4	Brisbane	12	565.00	47.0833
5	Toronto	14	563.00	40.2142
6	Berlin	12	529.00	44.0833
7	Lyon	10	437.00	43.70
8	Kyoto	10	412.00	41.20

מטלה 2 – יישומי כלים מתקדמים

View :

קוד ה-SQL לכתיבת כלי VIEW–

```
CREATE VIEW OrderDeliveryDetail AS

SELECT O.Order_ID , O.Email , D.Shipping_ID , D.Country , D.Arrival_Time , S.Search_DT AS Order_Placement_Timer

FROM Orders AS O JOIN Deliveries AS D ON O.Order_ID LEFT JOIN Searches AS S ON S.Order_ID = O.Order_ID
```

קוד ה- SQL שמממש את כלי ה-VIEW:

```
SELECT *

FROM OrderDeliveryDetails
```

הגיון עסקי – המטרה הינה איחוד כלל הטבלאות הרלוונטיות למעקב אחר ביצוע הזמנה ועד הגעת המשלוח ללקוח ופרטי התקשרות של הלקוח.

צילום מסך מהפלט שמפיק הבסיס נתונים -

	Order_ID	Email	Shipping_ID	Country	Arrival_Time	Order_Placement_Time
1	300000	emma.cohen@gmail.com	4000	Australia	2025-07-28 00:00:00.000	2025-04-14 00:00:00.000
2	300001	liam.levi@gmail.com	4001	Spain	2025-07-22 00:00:00.000	2025-03-26 00:00:00.000
3	300002	ava.mizrahi@gmail.com	4002	USA	2025-07-03 00:00:00.000	2025-04-04 00:00:00.000
4	300003	noah.shalev@gmail.com	4003	Italy	2025-07-14 00:00:00.000	2025-03-07 00:00:00.000
5	300004	sophia.azulay@gmail.c...	4004	Netherl...	2025-07-17 00:00:00.000	2025-05-26 00:00:00.000
6	300005	ethan.peretz@gmail.com	4005	Canada	2025-07-14 00:00:00.000	2025-04-14 00:00:00.000
7	300006	mia.sharabi@gmail.com	4006	Italy	2025-07-02 00:00:00.000	2025-06-16 00:00:00.000
8	300007	oliver.dahan@gmail.com	4007	Spain	2025-07-21 00:00:00.000	2025-05-04 00:00:00.000
9	300008	isabella.ben@gmail.com	4008	Australia	2025-08-01 00:00:00.000	2025-04-03 00:00:00.000
10	300009	james.golan@gmail.com	4009	France	2025-07-10 00:00:00.000	2025-03-24 00:00:00.000
11	300010	amelia.tzur@gmail.com	4010	Germa...	2025-07-17 00:00:00.000	2025-03-11 00:00:00.000
12	300011	benjamin.r@gmail.com	4011	Italy	2025-07-24 00:00:00.000	2025-04-01 00:00:00.000
13	300012	ella.shalom@gmail.com	4012	Spain	2025-07-05 00:00:00.000	2025-05-24 00:00:00.000
14	300013	lucas.danino@gmail.com	4013	USA	2025-07-22 00:00:00.000	2025-06-07 00:00:00.000
15	300014	lily.bashari@gmail.com	4014	Spain	2025-07-22 00:00:00.000	2025-05-07 00:00:00.000

פונקציות (Functions) :

פונקציה ראשונה פונקציה סקלרית –

קוד ה-SQL לכתיבת הפונקציה –

```
CREATE FUNCTION fn_GetDeliveryDurationDays (@Shipping_ID INT)

RETURN INT AS BEGIN

DECLARE @Duration INT

SELECT @Duration = DATEDIFF(DAY , Order_Placement_Time , Arrival_Time)

FROM OrderDeliveryDetails

WHERE Shipping_ID  = @Shipping_ID

RETURN @Duration

END
```

קוד ה- SQL שמממש את כלי ה-פונקציה:

```
SELECT Order_ID , Email , Order_Placement_Time , Arrival_Time

        dbo.fn_GetDeliveryDurationDays(Shipping_ID) AS Delivery_Duration_Days

FROM OrderDeliveryDetail

WHERE Order_Placement_Time IS NOT NULL AND Arrival_Time IS NOT NULL
```

הגיון עסקי – מטרת הפונקציה הינה חישוב הזמן שעבר בין ביצוע ההזמנה על ידי הלוקח ועד זמן ההגעה של המשלוח.

צילום מסך מהפלט שמפיק הבסיס נתונים -

	Order_ID	Email	Order_Placement_Time	Arrival_Time	Delivery_Duration_Days
1	300000	emma.cohen@gmail.com	2025-04-14 00:00:00.000	2025-07-28 00:00:00.000	105
2	300001	liam.levi@gmail.com	2025-03-26 00:00:00.000	2025-07-22 00:00:00.000	118
3	300002	ava.mizrahi@gmail.com	2025-04-04 00:00:00.000	2025-07-03 00:00:00.000	90
4	300003	noah.shalev@gmail.com	2025-03-07 00:00:00.000	2025-07-14 00:00:00.000	129
5	300004	sophia.azulay@gmail.c...	2025-05-26 00:00:00.000	2025-07-17 00:00:00.000	52
6	300005	ethan.peretz@gmail.com	2025-04-14 00:00:00.000	2025-07-14 00:00:00.000	91
7	300006	mia.sharabi@gmail.com	2025-06-16 00:00:00.000	2025-07-02 00:00:00.000	16
8	300007	oliver.dahan@gmail.com	2025-05-04 00:00:00.000	2025-07-21 00:00:00.000	78
9	300008	isabella.ben@gmail.com	2025-04-03 00:00:00.000	2025-08-01 00:00:00.000	120
10	300009	james.golan@gmail.com	2025-03-24 00:00:00.000	2025-07-10 00:00:00.000	108
11	300010	amelia.tzur@gmail.com	2025-03-11 00:00:00.000	2025-07-17 00:00:00.000	128
12	300011	benjamin.r@gmail.com	2025-04-01 00:00:00.000	2025-07-24 00:00:00.000	114
13	300012	ella.shalom@gmail.com	2025-05-24 00:00:00.000	2025-07-05 00:00:00.000	42
14	300013	lucas.danino@gmail.com	2025-06-07 00:00:00.000	2025-07-22 00:00:00.000	45
15	300014	lily.bashari@gmail.com	2025-05-07 00:00:00.000	2025-07-22 00:00:00.000	76

פונקציה שנייה פונקציה טבלאית –

קוד ה-SQL לכתיבת הפונקציה –

```
CREATE FUNCTION fn_SalesByCountryForCategory (@Category VARCHAR(50))

RETURNS TABLE AS RETURN (

SELECT D.Country , SUM(SIO.Quantity) AS Total_Sold

FROM SockInOrder AS SIO JOIN Socks AS S ON SIO.Item_Number  = S.Item_Number JOIN Deliveries AS D
ON SIO.Order_ID = D.Order_ID

WHERE S.Category = @Category

GROUP BY D.Counrty

)
```

קוד ה- SQL שמממש את כלי ה-פונקציה:

```
SELECT *

FROM dbo.fn_SalesByCountryForCategory('customized style')

ORDER BY Total_Sold DESC
```

הגיון עסקי –

הפונקציה מציגה עבור קטגורית עיצוב גרביים מסויימת שנבחר את כמות המכירות שלה בחלוקה למדינות

צילום מסך מהפלט שמפיק הבסיס נתונים -

	Country	Total_Sold
1	Japan	18
2	Italy	16
3	France	15
4	Canada	11
5	Germ...	9
6	Israel	9
7	Spain	9
8	USA	8
9	Austr...	6
10	Nethe...	4

Trigger :

קוד ה – SQL –

יצירת טבלה חדשה שתכיל את הפרטים הנדרשים:

```
CREATE TABLE CreditCardsExpiringSoon (

Card_Number CHAR (16) PRIMARY KEY ,

Expiration_Date DATE

Email VARCHAR(50)

Logged_At DATETIME DEFAULT GETDATE()

)
```

קוד ה- טריגר –

```
CREATE TRIGGER trg_LogExpiringCards ON Credit_Cards

AFTER INSERT , UPDATE AS BEGIN

INSERT INTO CreditCardsExpiringSoon (Card_Number , Expiration_Date , Email)

SELECT CC.CC_Number , CC.Expiration_Date , A.Email

FROM inserted CC JOIN Orders AS O ON CC.CC_Number = O.CC_Number JOIN Accounts AS A ON O.Account_Number = A.Account_Number

WHERE CC.Expiration_Date BETWEEN CAST (GETDATE() AS DATE) AND DATEADD(YEAR , 1 , GETDATE())

AND NOT EXISTS (

SELECT 1

FROM CreditCardsExpiringSoon AS E

WHERE E.Card_Number = CC.CC_Number

)

END
```

קטע ה – SQL של פעולה ה – UPDATE המקוננת המתאימה להפעלת הטריגר –

```
UPDATE Credit_Cards

SET Expiration_Date = CAST(DATEADD(MONTH , 6 , GETDATE()) AS DATE)

WHERE CC_Number = '5326103312341234'

AND EXISTS (

SELECT 1

FROM Orders AS O JOIN Searches AS S ON S.Order_ID = O.Order_ID

WHERE O.CC_Number = '5326103312341234'

AND S>Search_DT >= DATEADD(MONTH , -3 , GETDATE())

)
```

הגיון עסקי – ביצוע מעקב אחר לקוחות פעילים שתאריך תפוגת האשראי שלהם עתיד להגיע בשלושת החודשים הקרובים על מנת שהלקוחות יהיו ערים ויעדכנו את פרטי האשראי שלהם לפני פוג תוקפו.

הדגמה חיה תתבצע בעת מפגש ההגנות 😊

פרוצדורה שמורה (Stored Procedure) :

קוד ה – SQL – לכתיבת הפרוצדורה:

```
CREATE PROCEDURE CreateLateDeliveryCustomers AS

BEGIN

IF OBJECT_ID('LateDeliveryCustomers','U') IS NOT NULL

DROP TABLE LateDeliveryCustomers


CREATE TABLE LateDeliveryCustomers (

Order_ID INT ,

Email CARCHAR(100)

)


INSERT INTO LateDeliveryCustomers(Order_ID , Email)

SELECT Order_ID , Email ,

FROM OrderDeliveryDetail

WHERE dbo.fn.GetDeliveryDurationDays(Shipping_ID)>100 AND Order_Placement_Time IS NOT NULL
AND Arrival_Time IS NOT NULL

END


EXECUTE CreateLateDeliveryCustomers
```

הגיון עסקי – המטרה היא לייצר טבלה חדשה של כלל הלקוחות שזמן ההגעה הכולל של ההזמנה הוא מעל 100 ימים על מנת "לפצות" אותם על הזמן הארוך של המשלוח. בעזרת הפרוצדורה אנשי המכירות/שיווק בכל ארבעה חודשים יכולים לבדוק סטטוס הזמנות שנמצאות באוויר ויוכלו בעזרת פרטי ההתקשרות לעדכן לקוחות/לשלוח קופון אם ההזמנה חרגה/הנחה וכדומה

הדגמה חיה תתבצע בעת מפגש ההגנות 😊

מטלה 3 – כלים להצגת נתונים :

יוצגו בעת מפגש ההגנות על בסיס קבצי Power BI 😊

מטלה 4 – אופטימיזציה של שאילתות באמצעות Generative AI

מטלה 1 חלק sql.1
File

אוקיי אני רוצה עכשיו להעלות לך את קובץ ה-SQL שכולל את כל השאילתות שלי, עוברים לחלק 4 של שיפור זמני הריצה של השאילתות שלי ואני אשמח שתצביע לי על שאילתות שכתבנו שאפשר לשפר להם את זמני הריצה

מצוין – עברתי על קובץ ה־SQL המלא שלך. הנה ניתוח מקצועי של אילו שאילתות כדאי לשפר בביצועים, מה הסיבה לכך, ואיך אפשר לגשת לשיפור שלהן במסגרת חלק 4 של הפרויקט (אופטימיזציה עם Generative AI):

—

✅ שתי שאילתות שנבחרות לשיפור זמן ריצה:

1️⃣ השאילתה לחישוב אחוז שינוי החיפושים במרץ (Change_March_vs_Avg)

זוהי השאילתה הזאת:

Edit Copy sql

```sql
SELECT
    (SELECT COUNT(*)
     FROM Sock_Searches
     WHERE MONTH(Search_DT) = 3 AND YEAR(Search_DT) = 2025) AS March_Searches,

    (SELECT
        COUNT(*) / COUNT(DISTINCT MONTH(Search_DT))
     FROM Sock_Searches
     WHERE YEAR(Search_DT) = 2025
    ) AS Avg_Searches_Per_Month,

    CASE
        WHEN (
            (SELECT COUNT(*) ... ) - (SELECT COUNT(*) / COUNT(DISTINCT MONTH(...)))
        ) >= 0 THEN ...
```

🛠️ למה כדאי לשפר?

- אותה שאילתה (SELECT COUNT(*) FROM Sock_Searches WHERE ...) מופיעה שוב ושוב.
- ביצוע החישובים הרבים שוב על אותן תוצאות גורם לביצועים איטיים, במיוחד אם הטבלה גדולה.
- ניתן לשלב את כל החישובים ב־CTE אחד או אפילו בטבלה זמנית, ואז לבצע את ה־SELECT הסופי – כך מחזיקים את הנתונים פעם אחת בלבד.

2 פונקציית חלון מספר 2: ניתוח חיפושים חודשיים עם LAG ו־SUM

sql Edit Copy

```sql
SELECT
    S.Item_Number,
    ...,
    LAG(COUNT(*)) OVER (...) AS Prev_Month_Searches,
    ...
FROM
    Sock_Searches SS
JOIN Socks S ...
GROUP BY S.Item_Number, YEAR(...), MONTH(...)
```

🛠️ למה כדאי לשפר?

שימוש ב־COUNT(*) בתוך פונקציית חלון – לא תמיד נתמך או אופטימלי.

SUM(COUNT(*)) OVER – עלול לגרום לקינון לא יעיל.

הפתרון: לשמור את תוצאות GROUP BY בסיסי ב־CTE → ואז להפעיל פונקציות חלון על תוצאה שכבר סוכמה.

אפשר גם לבנות טבלה זמנית של חיפושים לפי חודש ולבצע עליה את הפעולות – כך מקטינים משמעותית את כמות החישוב בכל שלב.

—

🎁 הצעה לבונוס חכם:

תוכל לבקש מ־ChatGPT (או ממני) לכתוב עבורך גרסה אופטימלית יותר של כל אחת מהשאליתות האלו ולצרף את ההשוואה לקובץ ההגשה, כולל תיעוד זמן ריצה (או לפחות השוואת ביצועים תאורטית).

רוצה שאכתוב עבורך עכשיו את הגרסאות האופטימליות לשתי השאילתות האלה כדי שתוכל להציג שיפור בזמן הריצה?

אוקיי אני מאוד אוהב את הרעיונות שלך אני אשמח שתשלח לי גרסאות משופרות מבחינת זמני ריצה שמפיקות פלט זהה לחלוטין אבל בתוצאות זמני ריצה טובות יותר

מצוין – הנה גרסאות משופרות לשתי השאילתות שבחרנו, שמפיקות את אותו הפלט אך צפויות לרוץ בצורה יעילה יותר מבחינת ביצועים.

🔧 הרעיון המרכזי: להימנע משאילתות חוזרות זהות, להשתמש ב־CTE אחד שכולל את כל הנתונים הבסיסיים, ולבצע חישובים עליו.

—

✅ שיפור 1: שינוי אחוז החיפושים בחודש מרץ לעומת ממוצע חודשי בשנת 2025

🎯 הפלט נשאר זהה: כמות חיפושים במרץ, ממוצע חודשי, אחוז שינוי

👎 גרסה מקורית – עם 6 שאילתות מקוננות

✅ גרסה משופרת – עם CTE שמחשב הכול פעם אחת

sql

Edit Copy

```sql
with MonthlySearches AS (
    SELECT
        MONTH(Search_DT) AS MonthNum,
        COUNT(*) AS SearchCount
    FROM Sock_Searches
    WHERE YEAR(Search_DT) = 2025
    GROUP BY MONTH(Search_DT)
),
Summary AS (
    SELECT
        SUM(CASE WHEN MonthNum = 3 THEN SearchCount ELSE 0 END) AS March_Searches,
        AVG(SearchCount * 1.0) AS Avg_Searches_Per_Month
    FROM MonthlySearches
)
SELECT
    March_Searches,
    Avg_Searches_Per_Month,
    CASE
        WHEN Avg_Searches_Per_Month = 0 THEN 'N/A'
        WHEN March_Searches >= Avg_Searches_Per_Month THEN
            '+' + CAST(FLOOR(100.0 * (March_Searches - Avg_Searches_Per_Month) / Avg_Searches_Per_
        ELSE
            '-' + CAST(FLOOR(100.0 * ABS(March_Searches - Avg_Searches_Per_Month) / Avg_Searches_P
    END AS Change_March_vs_Avg
FROM Summary;
```

🧠 שיפור משמעותי:

- במקום להריץ 6 שאילתות על Sock_Searches, מריצים אחת בלבד עם GROUP BY ואז מחשבים על הנתונים המאוחסנים זמנית.

—

✅ שיפור 2: פונקציית חלון לניתוח מגמות חיפוש חודשי לגרביים

👎 הגרסה המקורית:

- מפעילה LAG ו־SUM על פונקציות COUNT(*) ישירות = לא יעיל

✅ הגרסה המשופרת:

- קודם כל יוצרים CTE עם סכומי החיפושים לפי גרב וחודש
- רק אז מפעילים עליהם LAG ו־SUM

sql

Edit Copy

```sql
unts AS (
    SELECT
        S.Item_Number,
        YEAR(SS.Search_DT) AS Y,
        MONTH(SS.Search_DT) AS M,
        COUNT(*) AS Monthly_Searches
    FROM Sock_Searches SS
    JOIN Socks S ON SS.Item_Number = S.Item_Number
    GROUP BY S.Item_Number, YEAR(SS.Search_DT), MONTH(SS.Search_DT)
),
Final AS (
    SELECT
        Item_Number,
        CONCAT(Y, '-', RIGHT('0' + CAST(M AS VARCHAR(2)), 2)) AS Search_Month,
        Monthly_Searches,
        LAG(Monthly_Searches) OVER (PARTITION BY Item_Number ORDER BY Y, M) AS Prev_Month_Searches,
        SUM(Monthly_Searches) OVER (PARTITION BY Item_Number ORDER BY Y, M ROWS BETWEEN UNBOUNDED F
    FROM MonthlyCounts
)
SELECT
    Item_Number,
    Search_Month,
    Monthly_Searches,
    Prev_Month_Searches,
    Monthly_Searches - Prev_Month_Searches AS Monthly_Change,
    Cumulative_Searches
FROM Final
ORDER BY Item_Number, Search_Month;
```

🧠 שיפור:

- רק CTE אחד מבצע GROUP BY בטבלה הכבדה
- פעולות החלון פועלות על תוצאה קטנה יותר → ריצה מהירה משמעותית

—

רוצה שאנסח לך גם הסבר מקצועי תמציתי על כל שיפור כדי שתוכל לצרף אותו למסמך ההגשה?

אני שמח על התוצאות שהפקת תודה! אני אשמח להסביר תמציתי שמסביר לי בדיוק איך זמני הריצה משתפרים בכדי שאוכל ללמוד ליישם זאת בעצמי להבא

איזה יופי שאתה רוצה להבין את זה לעומק – הנה הסבר תמציתי ומדויק על איך ולמה השיפורים שביצענו משפרים את זמני הריצה של השאילתות, עם תובנות שתוכל ליישם בהמשך:

—

✅ עיקרון 1: מניעת חישובים חוזרים

🧠 כלל זהב: אל תריץ שוב ושוב את אותה פונקציה או תת-שאילתה בתוך SELECT או CASE.

🔧 מה עשינו:
במקום לרוץ על Sock_Searches 6 פעמים עם COUNT(*) בתוך תתי SELECT, עשינו GROUP BY פעם אחת על כל החודשים ב־CTE אחד, ואז שלפנו את מה שצריך (חודש מרץ + ממוצע).

🚀 שיפור בפועל:

- צמצמנו את מספר הגישות לטבלה הכבדה מ־6 ל־1
- שרת SQL שומר תוצאה זמנית בזיכרון → זה מהיר משמעותית

✅ עיקרון 2: הפרדת שלבים – סיכום קודם, חישוב אחר כך

🧠 כלל: כשיש גם GROUP BY וגם פונקציות חלון – בצע את הסיכום (aggregation) קודם, ואז תפעיל עליו חלונות.

🔧 מה עשינו:

במקום להפעיל LAG על COUNT(*) (לא יעיל), יצרנו CTE בשם MonthlyCounts שעושה GROUP BY לפי גרב וחודש
→ ואז הפעלנו LAG ו־SUM על תוצאה שכבר סוכמה.

🚀 שיפור בפועל:

- SQL Server לא צריך לבצע אגרגציות בזמן הרצת כל שורת פונקציית חלון
- זה מוריד עומס חישובי מה־query plan ומשפר שימוש בזיכרון

—

✅ עיקרון 3: שימוש ב־CTE / טבלה זמנית לשמירת ביניים

🧠 כלל: כשיש לך שלבים מחושבים, שמור תוצאות זמניות (CTE, טבלה זמנית) במקום לחשב כל פעם מחדש.

💡 למה זה חכם:

- SQL יודע איך לאחסן זמנית את תוצאות ה־CTE בזיכרון (ולפעמים בדיסק), מה שחוסך זמן גישה
- קל יותר להבין, לבדוק ולשפר כל שלב בנפרד

🎓 לסיכום – שלושת העקרונות:

עיקרון	מתי ליישם
הימנע מחישובים חוזרים	כשאותו SELECT חוזר פעמיים או יותר
סכם קודם, חלון אחר כך	כשמשתמשים גם ב־GROUP BY וגם ב־OVER()
שמור תוצאות זמניות	כשיש שלבים חישוביים ביניים מורכבים

פרק שני - בונוס

מטלה 5 – יישום כלים נוספים בלימוד עצמי:

בחלק זה של הפרויקט בחרנו ללמוד באופן עצמאי וליישם את הכלים הבאים :

יישום כלי הטבלה הזמנית :

קוד ה – SQL –

```
CREATE PROCEDURE CreateLateDeliveryCustomers_Temp AS BEGIN

CREATE TABLE #LateDeliveryCustomers (

Order_ID INT ,

Email VARCHAR(100)

)

INSEST INTO #LateDeliveryCustomers (Order_ID , Email)

SELECT Order_ID , Email

FROM OrderDeliveryDetail

WHERE dbo.fn.GetDeliveryDurationDays(Shipping_ID) > 100 AND Order_Placement_Time IS NOT NULL AND Arrival_Time IS NOT NULL

SELECT *

FROM #LateDeliveryCustomers (אין צורך למחוק את הטבלה יותר מכיוון שהיא זמנית!!)

END


EXECUTE CreateLateDeliveryCustomers_Temp
```

הגיון עסקי – המטרה היית לקחת את הפרוצדורה שלנו ולא לשמור את הטבלה שמציגה את הלקוחות שיש לחלק להם בונוס על בסיס הנתונים מה שמכביד על בסיס הנתונים אלה להציג אותם אחת לשלוש חודשים לאנשי השיווק ולמחוק אותה מיד אחריי הסיום בשימוש בה כלומר לאחר שחולקו הבונוסים ללקוחות שהפיקה הפרוצדורה.

צילום מסך מהפלט שמפיק בסיס הנתונים –

	Order_ID	Email
1	300162	sockmatchx@gmail.com
2	300133	socksplash101@gmail.com
3	300180	sockkingdom@gmail.com
4	300107	snuglife22@gmail.com
5	300035	socksfan91@gmail.com
6	300137	footprintsx77@gmail.com
7	300095	patternsmiths@gmail.com
8	300177	funkyprints24@gmail.com
9	300124	funkyfitwear@gmail.com
10	300149	dotlover44@gmail.com
11	300131	feetscene44@gmail.com
12	300111	sockmate07@gmail.com
13	300102	walkonjoy99@gmail.com
14	300043	trendysocks2020@gmail....
15	300100	ilovestripes66@gmail.com

יישום כלי Merge :

קוד ה – SQL –

```
CREATE TABLE SockCategorySalesSummary (

Category_Name VARCHAR(20) PRIMARY KEY

Total_Sold_ INT NOT NULL

)

MERGE SockCategorySalesSummary AS Target

USING (

SELECT sc.Category_Name , SUM(so.Quantity) AS Total_Sold

FROM SockInOrder AS so JOIN Sock AS s ON so.Item_Number = s.Item_Number JOIN Sock_Categories AS sc ON s.Category = sc.Category_Name

WHERE s.Category IN ('Junk' , 'Food' , 'Christmas' , 'Funny' , Halloween' , ' Wine' , 'Winter')

GROUP BY sc.Category_Name

) AS Source ON Targer.Category_Name = Source.Category_Name
```

```
WHEN MATHCED THEN

UPDATE SET Targer.Total_Sold = Source.Total_Sold

WHEN NOT MATHCED THEN

INSERT ( Category_Name , Total_Sold)

VALUES (Source.Category_Name , Source.Total_Sold);
```

הרעיון העסקי –

במסגרת הניתוח העסקי שביצענו, זיהינו צורך בשמירה ועדכון קבוע של טבלת סיכום המרכזת את כלל המכירות לפי קטגוריית גרביים. טבלה זו מהווה בסיס לניתוחים שיווקיים חשובים, כמו זיהוי הקטגוריות הנמכרות ביותר, מדידת ביצועי קמפיינים ממוקדים, והבנת העדפות לקוח לאורך זמן.

לשם כך נעשה שימוש בפונקציית MERGE, המאפשרת לעדכן את טבלת הסיכום בצורה חכמה – כך ששורות קיימות יתעדכנו, ושורות חדשות ייווצרו אוטומטית עבור קטגוריות שטרם נרשמו בטבלה

צילום מסך מהפלט שמפיק בסיס הנתונים –

	Categor...	Total_Sold
▶	Christmas	28
	Funny	10
	Halloween	26
	Wine	9
	Winter	19
*	NULL	NULL